

Laboratorio 01: Búsqueda en espacio de soluciones

DANIEL ANTONIO SANTANILLA ARIAS

4 de agosto de 2026

1. INSTRUCCIONES

1.1. OBJETIVOS

Desarrollar competencias básicas para:

1. Modelar problemas para ser resueltos mediante técnicas de búsqueda en espacio de soluciones.
2. Implementar algoritmos de búsqueda sin adversario: A^*
3. Resolver problemas de búsqueda en espacio de soluciones.

1.2. ENTREGABLE

Reglas para el envío de los entregables:

- **Forma de envío:**

Este laboratorio se debe enviar únicamente a través de la plataforma [Moodle](#) en la actividad definida.

- **Archivo de entrega:**

Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio, revise [How to cite in Jupyter Notebooks](#) para citar cualquier recurso utilizado y ver la estructura del proyecto.

- **Nomenclatura para nombrar el archivo de entrega:**

El archivo deberá ser renombrado en mayúsculas con el siguiente formato:
`L01-{color_grupo}-IAIA-{semestre_academico}.zip`

Ejemplos:

`L01-AZUL-IAIA-2026-2.zip`

`L01-ROJO-IAIA-2026-2.zip`

Revise cuando lo requiera [1]

2. I. DESARROLLANDO: BÚSQUEDA SIN ADVERSARIO

2.1. A. MODELAR PROBLEMAS DE BÚSQUEDA

2.1.1. Modelen las estructuras de datos auxiliares para implementar búsquedas en espacio de soluciones.

Un Node del espacio de búsqueda que contenga:

- El *estado* (TState) que representa el *nodo* en el espacio de búsqueda
- El *nodo padre* (Node) como referencia al *nodo* que generó este nodo en el espacio de búsqueda
- La *acción* (TAction) que se aplicó al estado del *nodo padre* para generar este *nodo*
- El *costo del camino* (path_cost) que es el costo total desde el *nodo raíz* hasta este *nodo*
- Una *representación* para el nodo (representation() -> str)

Una Frontier que contenga:

- Defina si no hay nodos en la frontera (is_empty() -> bool)
- La función para eliminar el nodo superior de la frontera y devolverlo (pop() -> Node)
- La función para obtener el nodo superior de la frontera sin eliminarlo (top() -> Node)
- La función para agregar un nodo a la frontera (add(Node))

```
[ ]: # =====  
# Modele la estructura de datos para el espacio de búsqueda  
# =====  
  
class Nodo:  
  
    def __init__(self, estado, padre=None, accion=None, costo=0):  
        self.estado = estado  
        self.padre = padre  
        self.accion = accion  
        self.costo = costo  
  
    def __str__(self):  
        return f"Nodo(estado={self.estado}, accion={self.accion}, costo={self.  
↪costo})"
```

```
[ ]: # =====  
# Modele frontera del espacio de búsqueda  
# =====
```

2.1.2. Modelen la estructura de datos para un problema de búsqueda:

Un Problem que contenga:

- El estado inicial (initial -> TState)
- El/Los estado(s) objetivo(s) (goals -> Set[TState]), pueden ser uno o varios estados objetivo dependiendo del problema
- Si un estado es objetivo (is_goal(TState) -> bool)

- La función de transición de un estado al aplicar las acciones (`result_actions(TState) -> Set[TState]`)
- La función de costo de una acción (`action_cost(TState, TAction, TState_1) -> float`)
- La función de coste estimado de un estado (`heuristic(TState) -> float`)

```
[ ]: # =====
# Modele un problema del espacio de búsqueda genérico
# =====
```

2.2. B. IMPLEMENTAR

2.2.1. Implemente el algoritmo de búsqueda del mejor primero:

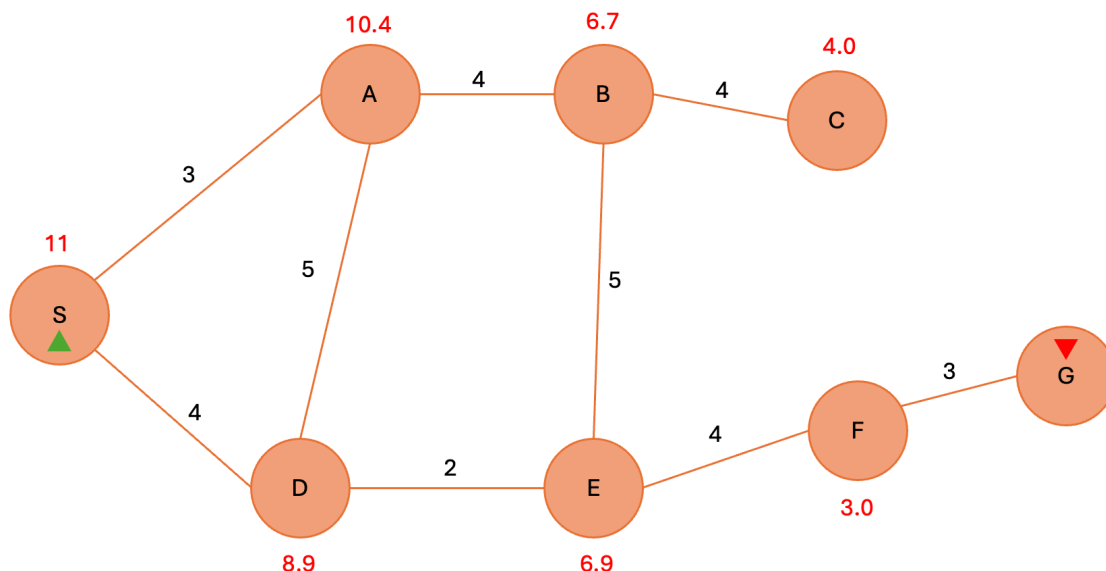
1. Implemente la indicación de fallo con un Node del espacio de búsqueda
2. Implemente la función para expandir un Node fronterizo `expand(Problem, Node) -> Sequence[Node]`
3. Implemente el algoritmo de búsqueda del mejor primero `best_first_search(Problem, evaluation_function) -> Node`

```
[ ]: # =====
# 1. Represente el nodo de fallo del espacio de búsqueda
# =====
```

```
[ ]: # =====
# 2. Implemente la función para expandir nodos fronterizos
# =====
```

```
[ ]: # =====
# 3. Implemente el algoritmo de búsqueda del mejor primero (Best-First Search)
# =====
```

2.2.2. Pruebe la implementación del algoritmo con el siguiente espacio de ejemplo:



1. Implemente la función que entregue la secuencia de acciones de un `Node` `path_actions(Node)` -> `Sequence[TAction]`
2. Modele la representación del problema de ejemplo.
3. Use el algoritmo de búsqueda del mejor primero para solucionar el problema de ejemplo
4. Presente la secuencia de acciones que soluciona el problema del nodo resultante una vez aplicado el algoritmo de búsqueda óptimo del mejor primero

Nota: Use el modelado del punto A como guía para representar y solucionar el problema de ejemplo

```
[ ]: # =====
# 1. Implemente la función para visualizar la secuencia de acciones de un `Node`
# usando `Node.representation()`
# =====
```

```
[ ]: # =====
# 2. Represente el problema de búsqueda de ejemplo
# con el modelado de problema de búsqueda genérico del punto A
# =====
```

```
[ ]: # =====
# 3. Use la implementación del algoritmo de búsqueda del mejor primero
# aplicándola al problema de búsqueda de ejemplo
# =====
```

```
[ ]: # =====
# 4. Use la función para visualizar la secuencia de acciones
# con el resultado del algoritmo de búsqueda del mejor primero
# aplicado al problema de búsqueda de ejemplo
# =====
```

2.2.3. Implemente el algoritmo de búsqueda A-Star (A^*):

1. ¿Qué necesita para que el algoritmo de búsqueda del mejor primero se comporte como A^* ?
2. Implemente el algoritmo de búsqueda A^*

1. Responda con conocimiento teórico la pregunta

```
[ ]: # =====
# 2. Implemente el algoritmo de búsqueda  $A^*$ 
# =====
```

3. II. SOLUCIONAR PROBLEMAS

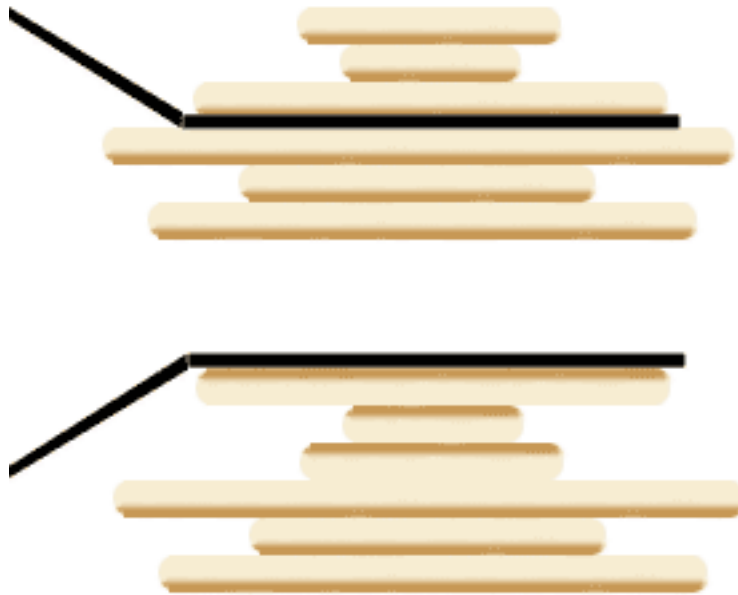
3.1. A. SOLUCIONANDO

Escoja uno de los siguientes problemas para solucionar: pancakes, granjero

3.1.1. A.1 Problema de pancakes

Dada una pila de pancakes de distintos tamaños, ¿puedes ordenarlas en una pila de tamaños decrecientes, de mayor a menor?

Tienes una espátula con la que puedes dar la vuelta a las i pancakes superiores. Esto se muestra a continuación para $i = 3$; en la parte superior, la espátula agarra las tres primeras pancakes; en la parte inferior, vemos cómo se dan la vuelta:



¿Cuántas vueltas se necesitarán para ordenar toda la pila?

Wiki: [Problema](#)

3.1.2. A.2 Problema del granjero



Un día, un granjero fue al mercado y compró un lobo, una oveja y una lechuga. Para volver a su casa tenía que cruzar un río. El granjero dispone de una barca para cruzar a la otra orilla, pero en la barca solo caben él y una de sus compras.

Si el lobo se queda solo con la oveja se la come, si la oveja se queda sola con la lechuga se la come.

El reto del granjero era cruzar él mismo y dejar sus compras a la otra orilla del río, dejando cada compra intacta. ¿Cómo lo hizo?

1. Presente el diseño del problema y del modelo (abstraction matemática)
2. Implemente los componentes necesarios para solucionar el problema
3. Use el algoritmo de búsqueda correspondiente para solucionar el problema
4. Presente la secuencia de acciones que soluciona el problema usando el algoritmo implementado

```
[ ]: # =====  
# 1. Represente el diseño del problema seleccionado  
# =====
```

```
[ ]: # =====  
# 2. Agregue lo necesario a su representación del problema  
# =====
```

```
[ ]: # =====  
# 3. Use el algoritmo de búsqueda A*  
# aplicándola al problema de seleccionado
```

```
# =====
```

```
[ ]: # =====  
# 4. Use la función para visualizar la secuencia de acciones  
# con el resultado del algoritmo de búsqueda A*  
# aplicado al problema de búsqueda seleccionado  
# =====
```

4. III. RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. ¿Cuál consideran fue el mayor logro? ¿Por qué?
4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Referencias

- [1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Education, 2021.